

Inteligência Artificial | FGA0021 | Turma 01 | 2026.1

Professor: Dr. Fabiano Araujo Soares

Aluno: Bruno Cruz Garcia Rosa

Matrícula: 222006605

CPS

1. Introdução

Em problemas de **Satisfação de Restrições (Constraint Satisfaction Problems – CSPs)**, o objetivo não consiste em encontrar o menor caminho ou minimizar custos, mas sim encontrar uma combinação válida de valores para um conjunto de variáveis, respeitando todas as restrições definidas.

Neste projeto, foi desenvolvido um sistema de coloração do mapa do Brasil utilizando técnicas de **CSP** e algoritmo de **Backtracking**.

O objetivo do problema consiste em atribuir cores aos estados brasileiros de forma que estados vizinhos nunca possuam a mesma cor.

Além da implementação tradicional do Backtracking, também foram utilizadas heurísticas clássicas de CSP:

- **MRV (Minimum Remaining Values);**
- **Grau da Variável.**

Essas heurísticas foram utilizadas para reduzir drasticamente a quantidade de chamadas recursivas e backtrackings necessários para encontrar uma solução válida.

Além disso, foi utilizada a biblioteca **Pygame** para a construção da interface gráfica e visualização da exploração dos algoritmos no labirinto, permitindo acompanhar visualmente a expansão dos nós e o caminho encontrado. Para execução correta do projeto, é necessária a instalação prévia da biblioteca Pygame através do comando:

- `pip install pygame`

ou:

- `pip install pygame-ce`

2. Definição do Problema

O problema abordado neste projeto consiste na coloração do mapa do Brasil utilizando técnicas de Problemas de Satisfação de Restrições (**CSP**).

Cada estado brasileiro é tratado como uma variável do problema, enquanto as cores representam os possíveis valores atribuídos às variáveis.

A principal restrição do problema estabelece que dois estados vizinhos não podem possuir a mesma cor.

O projeto utiliza o Teorema das Quatro Cores, que afirma que qualquer mapa plano pode ser colorido utilizando no máximo quatro cores sem que regiões adjacentes compartilhem a mesma cor.

Dessa forma, o algoritmo deve encontrar uma combinação válida de cores para os 27 estados brasileiros respeitando todas as restrições de vizinhança do mapa.

2.1. Formalização do Problema:

- **Estado Inicial:** O estado inicial corresponde ao mapa completamente sem coloração, onde nenhum estado brasileiro possui cor atribuída.
- **Ações:** As ações disponíveis consistem na atribuição de uma das quatro cores possíveis para um estado ainda não colorido.
- **Objetivo:** O objetivo consiste em colorir todos os estados brasileiros garantindo que estados vizinhos nunca possuam a mesma cor.
- **Variáveis:** Cada estado do mapa é tratado como uma variável.
- **Domínio:** conjunto de cores possíveis (vermelho, verde, azul, amarelo)
- **Restrições:** As restrições do problema determinam que:
 - a. Estados adjacentes não podem compartilhar a mesma cor;
 - b. Cada estado deve possuir exatamente uma cor válida.

2.2. Complexidade do Ambiente

O problema de coloração de mapas possui alta complexidade combinatória, pois a quantidade de combinações possíveis cresce rapidamente conforme aumenta o número de variáveis e restrições do sistema. No projeto desenvolvido, cada um dos 27 estados brasileiros pode assumir uma entre quatro cores possíveis, gerando um grande espaço de busca.

Além disso, a presença de múltiplas restrições de adjacência entre estados aumenta significativamente a dificuldade do problema, especialmente quando o algoritmo utiliza busca sem heurísticas.

Sem otimização, o algoritmo pode realizar milhares ou até centenas de milhares de chamadas recursivas até encontrar uma solução válida, dependendo da ordem de exploração das variáveis e das cores.

Por esse motivo, heurísticas como MRV e Grau da Variável se tornaram fundamentais para reduzir o espaço de busca e melhorar significativamente o desempenho do algoritmo.

3. Algoritmos Desenvolvidos

3.1. Grafo do Mapa

O mapa do Brasil foi representado computacionalmente através de um grafo de adjacência. Cada estado brasileiro representa um vértice do grafo, enquanto as fronteiras entre estados representam as arestas.

Também foi utilizada uma estrutura contendo as posições bidimensionais dos estados no mapa em uma tela de 800 px x 1200 px, permitindo a renderização gráfica correta utilizando Pygame.

3.2. FloodFill

Para permitir a coloração visual completa dos estados no mapa, foi implementado um algoritmo **Flood Fill**.

Inicialmente, o algoritmo recebe um pixel inicial localizado dentro do estado. Em seguida, realiza uma expansão recursiva para todos os pixels vizinhos válidos até encontrar as bordas do mapa. O mapa é um png com o mapa do Brasil em branco, com os estados delimitados e definidos.

As bordas são detectadas através da função *is_border()*, responsável por identificar pixels escuros correspondentes às linhas divisórias do mapa.

Após o processamento, todos os pixels pertencentes a cada estado são armazenados em memória utilizando a função *generate_state_pixels()*, permitindo que a coloração ocorra de maneira extremamente rápida durante a execução do algoritmo CSP.

3.3. CSP e Backtracking

A resolução do problema foi realizada utilizando o algoritmo Backtracking. O Backtracking funciona tentando atribuir cores aos estados de forma incremental. Caso uma atribuição viole alguma restrição do problema, o algoritmo desfaz a escolha atual e retorna para tentar outra possibilidade.

A função *is_consistent()* é responsável por verificar se uma cor pode ser atribuída a determinado estado sem gerar conflito com seus vizinhos.

Durante a execução:

- Um estado é selecionado;
- Uma cor válida é atribuída;
- O algoritmo avança recursivamente;
- Caso ocorra conflito, é realizado backtracking.

Também foram implementadas heurísticas clássicas de **CSP**:

- **MRV (Minimum Remaining Values);**
- **Grau da Variável.**

A heurística **MRV**, definida em *select_unassigned_variable()*, seleciona o estado com menor quantidade de cores válidas restantes, reduzindo rapidamente o espaço de busca.

Já o **Grau da Variável** é utilizado como critério de desempate, priorizando estados com maior quantidade de vizinhos e, conseqüentemente, maior influência sobre o restante do problema. Sem heurísticas, os estados e cores são escolhidos aleatoriamente, fazendo com que a quantidade de chamadas recursivas varie drasticamente entre diferentes execuções.

Dentro da função há chamadas para realizar a animação na tela, definidas por *draw_callback()*, que recebe alguns parâmetros para mostrar estatísticas. Um dos parâmetros utilizado é a utilização ou não de heurísticas para demonstrar como o não uso dela aumenta drasticamente a dependência da ordem dada dos estados e cores. Quando não é utilizado heurísticas, é usado a função *get_draw_frequency()* para diminuir a frequência de desenho à medida que as chamadas recursivas aumentam, melhorando a visualização e velocidade da animação.

3.4. Pygame

A biblioteca Pygame foi utilizada para construção da interface gráfica e visualização da execução do algoritmo. A interface permite:

- Visualizar o mapa sendo colorido em tempo real;
- Acompanhar métricas do algoritmo;
- Comparar execuções com e sem heurísticas.

Também foram adicionados botões interativos para execução dos dois modos, com ou sem heurísticas, para uma melhor comparação.

3.5. Execução Principal (Main)

O arquivo principal do projeto é responsável por inicializar a aplicação e controlar a interação do usuário com o sistema. Inicialmente, a interface gráfica é criada utilizando a função *initialize()*.

Em seguida, o programa entra em um loop principal para realizar a execução do algoritmo e da animação.

Ao clicar nos botões da interface, o algoritmo Backtracking é executado com ou sem heurísticas, permitindo comparar diretamente o comportamento dos dois modos de resolução.

Após a conclusão da execução, o mapa permanece colorido na tela juntamente com as métricas obtidas pelo algoritmo.

Figura 1 - CSP sem heurísticas. A quantidade de backtracks aumenta significativamente a depender da ordem das variáveis inicial.

